On-Device Driving Behavior Classification for Small Commercial Fleets

CSCI 49000-AIT · Midterm Project Progress Report · Week 9

Repository: [PASTE YOUR GITHUB URL HERE]

Muhammad Arfeen Hassan — hassa101@purdue.edu · Individual project (approved)

1. Status Summary

The Week 9 deliverable is **met, three weeks late**. My proposal named it as "Week 9 deliverable: interim report with the baseline comparison table and per-fold results." Both exist, computed on real recordings rather than on anything I generated: twelve labelled driving sessions, 972,755 samples of 100 Hz accelerometer and gyroscope signal, roughly 2.7 hours of driving, windowed into 9,530 examples and evaluated under leave-one-day-out. The learned model reaches a pooled macro-F1 of 0.787 against 0.283 for the fixed-threshold baseline it has to beat and 0.429 for a majority-class floor.

What works end to end today is the whole path from raw per-sample readings to a per-fold metrics table: a loader for the dataset, segmentation into overlapping windows with fill and label-purity checks, 50 features per window, leave-one-group-out cross-validation, a threshold baseline that tunes its own cut point on the training folds, and a majority-class floor. What does not exist is the trip-level aggregation stage, any on-device export, and any latency or model-size measurement.

The honest part of this status is the schedule rather than the artifacts. The Week 6 task of obtaining data was not done when planned, and Weeks 7 and 8 therefore had nothing to act on; the pipeline, the dataset work and this report were compressed into the final day. The code is real and reproducible, but the three-week slip is real too, and Section 4 sets out what I have changed so that Weeks 10 through 13 do not repeat it.

2. Background Research

Window length, and why mine changed. My proposal committed to thirty-second windows with fifty percent overlap, chosen because drowsiness is a slow phenomenon and I assumed the window had to be long enough to contain it. Banos et al. [1] tested window lengths systematically for inertial activity recognition and found the one-to-two second interval gives the best trade-off between recognition speed and accuracy, with longer windows not improving matters. That changed the decision: the primary window is now two seconds, with thirty retained as a comparison arm.

Having both arms let me check the claim on my own data rather than take it on authority, and the result is more interesting than a simple confirmation. Pooled macro-F1 is 0.787 at two seconds against 0.781 at thirty, which is a tie; what separates them is that thirty-second windows yield 486 examples from the same recordings against 9,530 at two seconds, and their mean per-fold score is lower and more variable (0.607 against 0.640). The short window is the better choice here because it extracts twenty times more supervision from a fixed quantity of driving, not because it classifies any single moment better. The reason I originally wanted long windows survives in a different form: slow behavior is now meant to be captured by aggregating short-window labels over a trip, which is the step my proposal already described for the dispatcher summary.

The evaluation split, which held up. My proposal chose to split by driver rather than randomly over windows. Saeb et al. [2] quantify what the random split costs: across the studies they surveyed, record-wise cross-validation reported a median classification error of 5.80 percent against 14.15 percent for subject-wise validation, and on a public activity dataset record-wise error sat near two percent regardless of training-set size while subject-wise stayed at seven to nine percent. A split that ignores

subject identity reports less than half the true error. The paper changed one implementation detail: my windower now discards any window whose samples are less than ninety percent one label, so a window straddling a behavior change cannot be handed an arbitrary label. It also set the grouping I use below, where both sessions recorded on one day are held out together, since they share a driver, a vehicle and a phone placement.

Dataset documentation. The dataset I am now using [3] is a public smartphone recording set with per-day folders whose names end in R or S for rash and safe driving. Reading its structure changed the problem definition: it carries no drowsy class at all, so the three-class problem my proposal specified cannot be evaluated on it. Section 3 records that change and Section 4 treats it as a scope decision rather than a deferral.

3. Progress and Code

The repository is linked in the header. Under *src/* are the loader, the windower, the feature extractor and the evaluation harness; under *experiments/* are three runnable scripts, and *results/* holds their CSV outputs. **Running end to end:** everything from loading the raw sensor files to the per-fold table. **Scaffolded or absent:** trip-level aggregation, TensorFlow Lite export, and on-device measurement.

Against my proposal's pipeline the DIKW layers now stand as follows. **Data** is reached: twelve labelled sessions load and are time-corrected. **Information** is reached: 9,530 windows with 50 features each. **Knowledge** is reached: the model is trained and evaluated honestly across held-out days. **Wisdom** is not reached; nothing yet turns window labels into a trip summary or an action.

Preliminary results. Leave-one-day-out over seven days, two-second windows, accelerometer channels only:

Model	Pooled acc.	Pooled macro-F1	Mean fold F1	Fold std	Worst fold
Gradient-boosted trees	0.853	**0.787**	0.640	0.263	0.400
Fixed-threshold baseline	0.734	0.283	0.336	0.115	0.226
Majority class	0.750	0.429	0.580	0.291	0.344

Table 1. 9,530 real windows, 2,381 rash and 7,149 safe, from 972,755 samples of driving.

The headline comparison my proposal promised now has an answer on real data: the learned model beats the fixed-threshold detector that represents current practice, 0.787 against 0.283, and beats the majority-class floor of 0.429. The pooled confusion matrix shows where it stands: of 2,381 rash windows it recovers 1,411 (recall 0.593), and of 7,149 safe windows it keeps 6,721 (recall 0.940), so the model is conservative, missing roughly four rash windows in ten rather than crying wolf on safe driving. For a coaching application that is the better direction to err in, but it is also the number that most needs improving by Week 13.

Fold variance, and the reason for it. The mean per-fold macro-F1 of 0.640 sits well below the pooled 0.787, with a standard deviation of 0.263 across seven folds (0.964, 0.994, 0.771, 0.400, 0.479, 0.447, 0.423 for days one through seven). This is not noise. Three of the seven days have essentially no rash driving in them: days five and seven contain zero rash windows and day three contains three. When such a day is held out, macro-F1 is computed over a single present class and is close to meaningless. My proposal predicted that six or seven groups would produce unstable folds; the instability is real, but its cause is label coverage rather than sample size, and reporting per-fold results is what made it visible.

A leak I found and removed. Two of the twelve sessions, Day-2R and Day-6R, ship no gyroscope file, and my loader zero-fills those channels. Both are rash sessions, so a constant-zero gyroscope is a perfect giveaway for the positive class, and gyroscope features let the model exploit it: including them raises pooled macro-F1 from 0.787 to 0.808. That 0.021 gain is not driving signal, it is file availability, so Table 1 reports the accelerometer-only configuration and the repository keeps both so the difference stays auditable.

A data-quality problem worth recording. The millisecond column in these files cannot be used as written. One session carries 111,212 rows under only 7,663 distinct timestamps: most are about 10 ms apart, which is the genuine 100 Hz acquisition, but some repeat up to 276 times and the largest gap between distinct stamps is 260 seconds. The logger stalls and then writes its backlog under a single stamp. Taken at face value the column makes 93 percent of consecutive samples look simultaneous and collapses an eighteen-minute recording into about a hundred usable windows. The loader rebuilds the clock from acquisition order at the rate the file's own wall-clock column confirms, and cuts the recording at every stall so that no window spans one.

Design changes. Two. The window length moves from thirty seconds to two, for the reasons in Section 2. More consequentially, the problem is now two-class, rash against safe, rather than the three-class normal, aggressive and drowsy my proposal specified, because the available labelled data contains no drowsy sessions. No change to devices: the target remains a phone-class processor, and nothing measured so far bears on that choice.

4. Schedule, Risks, and Mitigations

Schedule. Planned against actual:

Planned (from the proposal)	What actually happened
Week 6: request dataset access, set up repository, load and plot raw trips	Not done in Week 6. Repository created in Week 9.
Week 7: windowing, filtering, feature extractor, synthetic dataset as stand-in	Done in Week 9. Filtering still not implemented.
Week 8: threshold and majority-class baselines, first leave-one-driver-out run	Done in Week 9, on real data, grouped by day rather than driver.
Week 9: interim report with baseline comparison table and per-fold results	Met: Table 1 and the seven per-day folds, on 2.7 hours of real driving.

Everything slipped by about three weeks, and the cause was the unobtained dataset rather than any technical obstacle: once the first task was skipped, the weeks that depended on data had nothing to act on. The correction is concrete rather than a promise to work harder: the data is now in hand and the loader is written, which is the dependency that blocked everything else.

Revised plan through Week 13. Week 10: raise rash recall from 0.593, starting with class weighting for the three-to-one imbalance and a per-day error analysis of the three folds that scored below 0.45; add the low-pass filtering step that was planned for Week 7 and never written. Week 11: the trip-level aggregation stage, turning window labels into a session summary, which is the wisdom layer and the part a dispatcher would actually see. Week 12: model size and single-window latency on the development machine, with the TensorFlow Lite export [5] attempted if the earlier weeks hold; this week also carries the slack. Week 13 delivers what the proposal named: "Week 13 deliverable: final report, demonstration and the reproducible repository." As an individual project every component is mine: data, pipeline, evaluation and measurement.

Mitigations already taken. My first proposal risk was data access, and it materialized, though not in the form I predicted: not a license delay, but my own failure to start it. The planned mitigation had two halves. The half I executed was keeping the pipeline dataset-agnostic, writing every module against a generic long dataframe of timestamp, signal, label and group columns; that is why adding the real dataset took one loader module rather than a rewrite, and it is the reason Table 1 exists. The half I did not execute was obtaining data early, which is what cost three weeks. I also checked the proposal's named fallback before relying on it and measured that it could not carry the modelling claim: on that synthetic trip-level file, linear regression predicts the target at R-squared 0.9339 while gradient boosting reaches 0.9295, so a flexible model cannot beat a straight line and the target is a formula plus noise. It also has no per-sample timestamp and no driver identifier. Ruling it out in Week 9 rather than Week 13 is the mitigation that mattered most.

My second risk was that a small number of groups would make leave-one-out folds unstable. It materialized and is quantified above: fold macro-F1 ranges from 0.400 to 0.994 with a standard deviation of 0.263. The planned mitigation, reporting per-fold rather than a single average, is what exposed the cause, which turned out to be label coverage rather than group count.

Additional mitigations needed. Three risks surfaced while implementing. First, three of seven days contain no rash driving, so those folds cannot produce a meaningful macro-F1; the mitigation is to report pooled metrics alongside per-fold ones and to state explicitly which folds are single-class, rather than averaging them into a headline number. Second, missing sensor files can become labels, as the zero-filled gyroscope did; the mitigation is already in the code, which records per session whether each sensor was present, and the rule that any configuration including a zero-filled channel is reported separately. Third, the dataset has no drowsy sessions, so the three-class problem cannot be evaluated; rather than quietly dropping a class I promised, the final report will present the two-class problem as the result and name the missing class as a limitation of the data.

Minimum viable result, restated and narrowed. My proposal's minimum was a complete reproducible pipeline from raw inertial signal to window labels, a leave-one-driver-out benchmark table against the threshold baseline, and measured model size and inference latency. The first two are now met, with grouping by day rather than by driver, which is a weaker control and is named as such. I am narrowing the third: size and latency will be measured on the development machine, and the on-phone measurement becomes a stretch goal rather than a requirement, because it depends on an export path I have not started and I would rather commit to what the remaining weeks can actually deliver.

5. References

[1] Oresti Banos, Juan-Manuel Galvez, Miguel Damas, Hector Pomares, and Ignacio Rojas. 2014. Window Size Impact in Human Activity Recognition. *Sensors* 14, 4 (2014), 6474-6499. https://doi.org/10.3390/s140406474

[2] Sohrab Saeb, Luca Lonini, Arun Jayaraman, David C. Mohr, and Konrad P. Kording. 2017. The need to approximate the use-case in clinical machine learning. *GigaScience* 6, 5 (2017), 1-9. https://doi.org/10.1093/gigascience/gix019

[3] Driver Behavior Detection Using Smartphone (dataset). Mendeley Data. Retrieved October 22, 2026 from https://data.mendeley.com/datasets/9vr83n7z5j

[4] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825-2830.

[5] Google. 2026. LiteRT (TensorFlow Lite) Model Conversion. Retrieved October 22, 2026 from https://ai.google.dev/edge/litert/models/convert

[6] Android Open Source Project. 2026. Sensors Overview. Retrieved October 22, 2026 from https://developer.android.com/guide/topics/sensors/sensors_overview

[7] sonalshinde123. Eco-Driving Behavior Dataset (synthetic, 30,000 records). Kaggle. Retrieved October 2, 2026 from https://www.kaggle.com/datasets/sonalshinde123/eco-driving-behavior-dataset